

ROCKE-3D ON PYTHON/JAX

Approach and Results of a Fortran-to-JAX Port

What an AI coding agent could do with part of NASA GISS's ROCKE-3D climate model: translate it, check it against the real model, and make it fast. Written as the starting point for repeating the exercise with a full-fidelity port in a separate branch.

356×

faster than Fortran on a GPU, for the surface-physics slice that was ported

5

stages measured on the same real P2SAoM40 restart data

0.965

spatial correlation with real model temperatures; kernels match Fortran to rounding

~9%

of Fortran's runtime is covered by that slice: scope is the caveat

How the Exercise Was Run

1 Fix the reference

Real Fortran ROCKE-3D, configuration P2SAoM40 (72×46 grid, 40 layers), a real restart file, and per-routine costs read from the real run log.

2 Port module by module

An AI agent translated the column-independent physics (PBL, dry convection, surface, radiation stand-in) to Python/JAX.

3 Validate against Fortran

Compiled Fortran test drivers run on identical inputs; JAX and Fortran outputs compared (CPU and GPU).

4 Chain them into a driver

PBL + surface + ground + simplified radiation, in the real per-step call order, started from the real restart.

5 Benchmark five stages

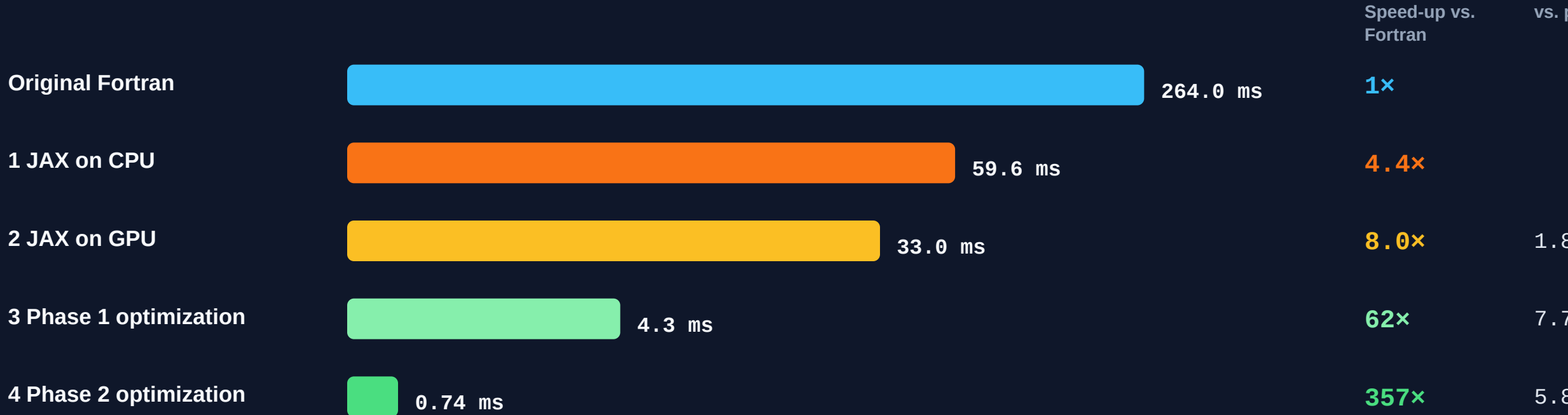
Fortran, JAX/CPU, JAX/GPU, then two optimization rounds, all timed on the same step of the same data.

6 Optimize by profiling

Profile, remove the biggest cost, re-check accuracy, re-measure. Repeat. Verify any dramatic result before reporting it.

RESULTS

Five Stages, from Fortran to 356× Faster



CPU-only tells the same story

Phase 1 → Phase 2 on CPU: 17.9 ms → 1.9 ms (9.5×). CPU stage 1 (≈60 ms) is an older, cross-node measurement.

Two numbers for Phase 2

0.74 ms only when the state stays on the GPU across steps. One step per call: 3.6 ms (1.2× over Phase 1).

Why 8× is not 356×

The big wins came from software structure (job count, data movement). The first GPU port alone gained under 2× (cross-node).

What Each Stage Is, in Plain Terms

Original Fortran

264 ms

The real NASA model code, run on an ordinary processor. It is the reference for both speed and answers.

1 JAX on CPU

≈60 ms

An AI agent rewrote the physics in Python/JAX, a language built for fast array math. Same processor, first draft, not tuned.

2 JAX on GPU

33 ms

The same rewrite on a graphics processor made for thousands of parallel calculations. It gains only modestly because the work is split into many tiny GPU jobs.

3 Phase 1

4.3 ms

About 33 tiny GPU jobs per step were merged into one, so the GPU spends its time computing instead of starting jobs.

4 Phase 2

0.74 ms

The model's data now stays on the GPU across many steps instead of being copied back and forth each step, constants are computed once, and the slowest routines were rewritten.

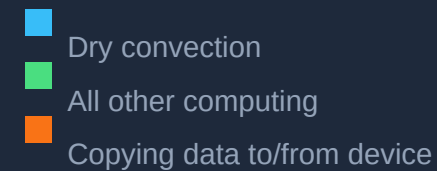
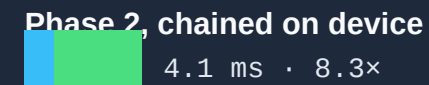
Where the Time Went

Before Phase 1: one step ≈ 55 ms



Every function was compiled but called separately from Python: 33 GPU/CPU jobs per step plus glue. Phase 1 fused these into one.

Phase 1 vs Phase 2 (stacked)



Dry convection 12.3 $\rightarrow$ 1.1 ms (11 $\times$) · other computing 14.7 $\rightarrow$ 3.1 ms (4.8 $\times$) · data copying 7.0 $\rightarrow$ 2.8 ms, now 41% of a single call, which is why keeping data on the device matters.

Did the Answers Stay the Same?

JAX vs. real Fortran (kernels)

PBL and dry convection, same inputs, CPU and GPU: agreement at floating-point level ($1e-9$ to $1e-3$ across fields).

Full step vs. real model data

Layer-1 air temperature vs. the real run's period-mean surface temperature: correlation 0.965, bias -1.9 °C. Unchanged by both optimizations.

Optimized vs. original JAX

Dry convection vs. an independent float64 reference: $< 5e-4$ K. Chained steps differ from Phase 1 at rounding level; a few marginal cells flip a branch in an iterative solver, exactly as they do when Phase 1 itself is nudged by one rounding step, so this is noise, not regression.

How this was checked

Independent float64 references over every stability branch; deliberately breaking the code to confirm the tests fail (mutation testing); a noise-floor test; comparison to real Fortran outputs. Two measurement bugs were found and fixed on the way.

What the Numbers Do Not Say

Coverage of the real model (Fortran run log)

Ported and timed: surface and ground physics, about 9% of Fortran's runtime (SURFACE 8.9%).

Radiation, 65.6% of runtime: a simplified stand-in here, excluded from the comparison.

Moist convection (9.9%) and atmospheric dynamics (8.5%): not ported. They need values from neighbouring grid cells, which does not vectorize as easily.

So **no whole-model speed-up is claimed.**

The ported slice does less physics than Fortran

One dominant surface type per cell instead of area-weighted sub-tiling; simplified turbulence; land hydrology (GHY), sea ice and lakes are placeholder code.

An earlier claim of “14 of 17 modules faithfully ported” was wrong: the GHY test compared a stub with itself and hardcoded outputs to zero.

A full-fidelity port would do more work and would likely show a smaller multiple. Whether a real speed-up survives is the question a full-fidelity branch answers.

Lessons for the Full-Fidelity Port

Do

- **Write the pass criteria first:** per module, which Fortran routine and what tolerance.
- **Derive every test driver from the real Fortran source;** never a hand-written stub.
- **Audit for “placeholder / simplified” before calling a module done.**
- **Time all stages on the same node, commit and method;** record the Fortran and first-JAX baselines first.
- **Force CPU in its own process;** synchronize before reading clocks; report single-call and chained.
- **Design for the GPU from day one:** layer-first arrays, state resident on the device, constants computed once, one compiled job per step.
- **Profile before optimizing;** verify any dramatic result (reproduce, sanity-check, state scope).

Avoid (each one happened here)

- **Self-referential tests:** a stub “reference” shared the port's simplifications and passed (GHY, SEAICE, LAKES, ATURB).
- **Incomplete reference code:** a NumPy stand-in omitted branches and showed diffs up to 1.26e4 until rebuilt from the real formulas.
- **A “CPU” baseline that silently ran on the GPU,** producing a false “no GPU benefit”.
- **Mixing nodes:** the ≈ 60 ms JAX/CPU figure came from another machine.
- **Array-layout footguns:** Fortran vs C ordering corrupted shared test inputs until an input round-trip check caught it.
- **Claiming whole-model results from a slice:** radiation alone is 66% of Fortran's time.
- Unit tests pass on synthetic values; only real restart data exposed a wind-speed bug that zeroed land fluxes.

Where Everything Lives

STATUS.md

Single source of truth: goal, accuracy, five-stage results, Round 2 optimization, simplifications, recommendation, gotchas.

README_GPU.md

How to run the benchmarks on the GPU cluster and which driver path to use.

RESTART_SESSION.md

Session-to-session hand-off notes and history.

p2saom40_driver.py · p2saom40_compare.py

Full-chain driver (single-call and chained device-resident) and the five-stage benchmark harness.

drycnv.py · pbl.py

The two kernels validated against Fortran and optimized in Round 2.

compare_fortran.f90 · compare_jax.py

Kernel-level Fortran-vs-JAX comparison; data in `compare_data/`.

tests/ · tests/test_round2_optimization*.py

115 regression tests including float64 references.

status_slides/

Status deck sources (deck.json, slides/*.html) and this deck's build script.